DAILY TASK ORGANIZER

Python Mini Project

A simple project for organizing daily tasks into two categories: **completed_tasks** and **incomplete_tasks**.

Project Type	Python Console Application
Main Purpose	Track daily tasks and separate them by completion status
Main Lists	completed_tasks, incomplete_tasks
Input	Task checklist and completion status
Output	Completed and incomplete task lists

1. Problem Statement

At the start of the day, a user creates a checklist containing the tasks they want to accomplish. At the end of the day, each task is reviewed. Tasks that were finished are placed in **completed_tasks**, while tasks that were not finished are placed in **incomplete_tasks**. The project provides a simple way to see the day's progress.

2. Objectives

1. Create a daily checklist.

2. Accept multiple tasks from the user.

3. Ask whether each task was completed.

4. Store finished tasks in completed_tasks.

5. Store unfinished tasks in incomplete_tasks.

6. Display a clear summary at the end of the day.

3. Working / Flow

START	Begin the program
STEP 1	Enter the number of tasks
STEP 2	Enter each task
STEP 3	Mark each task as completed or incomplete
STEP 4	Add completed tasks to completed_tasks
STEP 5	Add unfinished tasks to incomplete_tasks
END	Display the daily summary

4. Python Program

```
# Daily Task Organizer

completed_tasks = []
incomplete_tasks = []

print("===== DAILY TASK ORGANIZER =====")

n = int(input("Enter number of tasks for today: "))

tasks = []

for i in range(n):
    task = input(f"Enter task {i + 1}: ")
    tasks.append(task)

print("\n===== END OF DAY REVIEW =====")

for task in tasks:
    status = input(f'Did you complete "{task}"? (yes/no): ')

    if status.lower() == "yes":
        completed_tasks.append(task)
    else:
        incomplete_tasks.append(task)

print("\n===== DAILY SUMMARY =====")

print("\nCompleted Tasks:")
if completed_tasks:
    for task in completed_tasks:
        print("- " + task)
else:
    print("No completed tasks.")

print("\nIncomplete Tasks:")
if incomplete_tasks:
    for task in incomplete_tasks:
        print("- " + task)
else:
    print("No incomplete tasks.")

print("\nTotal completed:", len(completed_tasks))
print("Total incomplete:", len(incomplete_tasks))
```

Note: Save the program as **main.py** and run it using **python main.py**.

5. Sample Input and Output

```
===== DAILY TASK ORGANIZER =====
Enter number of tasks for today: 4

Enter task 1: Complete Python assignment
Enter task 2: Study Java
Enter task 3: Exercise
Enter task 4: Read a book

===== END OF DAY REVIEW =====
Did you complete "Complete Python assignment"? (yes/no): yes
Did you complete "Study Java"? (yes/no): yes
```

```
Did you complete "Exercise"? (yes/no): no
Did you complete "Read a book"? (yes/no): no

===== DAILY SUMMARY =====

Completed Tasks:
- Complete Python assignment
- Study Java

Incomplete Tasks:
- Exercise
- Read a book

Total completed: 2
Total incomplete: 2
```

6. Explanation of Important Terms

Term	Simple Meaning
completed_tasks	List that stores tasks finished during the day.
incomplete_tasks	List that stores tasks that were not finished.
tasks	List that temporarily stores the day's checklist.
append()	Adds a new task to a list.
input()	Takes information from the user.
if / else	Checks whether a task was completed.
len()	Counts the number of tasks in a list.

7. Advantages

• Very easy to understand and use.

• Helps organize daily work.

• Clearly separates finished and unfinished work.

• Uses basic Python concepts such as lists, loops, input, and conditions.

• Can be extended into a GUI or web application later.

8. Future Enhancements

• Add task priorities such as High, Medium, and Low.

• Save tasks permanently in a text file or database.

• Add task due dates and reminders.

• Create a graphical user interface using Tkinter.

• Create a web version using Flask or Django.

• Show daily and weekly completion statistics.

9. Conclusion

The Daily Task Organizer is a simple Python project that helps a user review their daily checklist. It separates tasks into completed_tasks and incomplete_tasks, making it easy to understand what was achieved and what still needs attention. The project is also a good beginner exercise for learning Python lists, loops, conditional statements, user input, and basic program organization.